Eric Romero
CSC 445
3/30/2026

Homework 8

Formulate the problem recursively. Write down a recursive formula or algorithm for the whole problem in terms of the answers to smaller subproblems. This is the hard part. A complete recursive formulation has two parts:

a. **Specification.** Describe the problem that you want to solve recursively, in coherent and precise English—not how to solve that problem, but what problem you're trying to solve. Without this specification, it is impossible, even in principle, to determine whether your solution is correct.

Given position i, the recursive problem to solve is to pretty-print optimally the words from i to n - 1. Assume the word at position i starts a new line. The goal is to choose a line break so that every line reaches a max length (L) and the weight of the value is minimized. This is found by applying the slack function and summing.

b. **Solution.** Give a clear recursive formula or algorithm for the whole problem in terms of the answers to smaller instances of exactly the same problem.

Consider 'OPT(i)' (recall the Dynamic Programming slides). In reference to the assignment spec, this is M. This represents the minimum total cost of pretty printing from i to n - 1. We will consider OPT(0).

If i = n, no words left. Cost is 0.

Base Case: OPT(n) = 0

For position i < n, next line can end at index j, where j >= i and words i to j fit the max line length (L).

The line with words i to j has a length of:

$$\sum_{k=i}^{j} w_k + (j - i)$$

'(j - i)': number of spaces between words
'w_k': in this case, represents a word

Slack is:

$$L - (\sum_{k=i}^{j} w_k + (j - i))$$

Consider the following from the spec:

K line breaks

Eric Romero
CSC 445
3/30/2026

Output is k lines
Slack of lines: $s_0, s_1, \ldots, s_k$

If we are trying to optimize this, for words in index i to j, where j >= i, and its the words that fit between the two indices, the formula is:

$$OPT(i) = MIN\left[f(L - (\sum_{k=i}^{j} w_k + (j - i)) + OPT(j + 1)\right]$$

The first line is fixed, and the following lines follow suit, where the second term ('OPT(j + 1)') is where we apply recursion.

Build solutions to your recurrence from the bottom up. Write an algorithm that starts with the base cases of your recurrence and works its way up to the final solution, by considering intermediate subproblems in the correct order. This stage can be broken down into several smaller, relatively mechanical steps:

a. **Identify the subproblems.** What are all the different ways your recursive algorithm can call itself, starting with some initial input?

Subproblems!

Base is OPT(0) → meaning the first line. Recursive calls move through the paragraph. Format of this would look like:
$OPT(0), OPT(1), \ldots, OPT(n)$

Because of this,
OPT(i) =minimal slack cost of formatting words from i to n, assuming i starts a new line, for for i in 0 <= i < n.

OPT(n) = empty suffix, cost is 0

There are n + 1 subproblems total where each subproblem is the start of a new line.

b. **Choose a memoization data structure.** Find a data structure that can store the solution to every subproblem you identified in step (a). This is usually but not always an array (possibly multidimensional).

Consider index i. Each subproblem is recognized by one weight.
Let M[i]: Minimum total slack cost for pretty printing suffix start at word i.

M[0] = 0: Base Case

Eric Romero
CSC 445
3/30/2026

1-D array is good to store optimal values. Another 1-D array should be used for rebuilding. We will call this solutions[i], which follows where the first line of the subproblem i ends. After the recurrence processes all n+1 options, this array will be used to gather the list of last word indices.

c. **Identify dependencies.** Except for the base cases, every subproblem depends on other subproblems—which ones? Draw a picture of your data structure, pick a generic element, and draw arrows from each of the other elements it depends on. This will help with the next step.

Dependencies for each subproblem at index i are the subproblems that remain after the first line is selected where if the first line uses the words from i to j, then the rest of the text has to begin at j + 1. Thus, the dependencies would look like for index i:
M[i] -> M[i+1] -> M[i + 2] -> … -> M[n]

d. **Find a good evaluation order.** Order the subproblems so that each one comes after the subproblems it depends on. You should consider the base cases first, then the subproblems that depends only on base cases, and so on, eventually building up to the original top-level problem.

Given the dependencies above, OPT(i) relies on subproblems with > i. The table will thus be completed right to left:

Base: OPT(n) = 0
Followed by: OPT(n - 1), OPT(n - 2), … OPT(0)

e. **Analyze space and running time.** The number of distinct subproblems determines the space complexity of your memoized algorithm. To compute the total running time, add up the running times of all possible subproblems, assuming deeper recursive calls are already memoized. You can actually do this immediately after step (a).

Time Complexity:

Considering OPT(i), the algorithm will start at index i and moves forward until reaching j, meaning the ending position of the line, while also considering it still fits on that one line.

In the worst case, the algorithm would check n - i possible weights for index j. The cost of the remainder from recursive calls is put into M[j + 1]. The algorithm has to maintain the current line length and slack, alongside any additional work from the considerations: O(n - i) for just one subproblem.

In total:

$$\sum_{i=0}^{n-1} O(n - i) = O(1 + 2 + \ldots + (n - 1) + n) = O(n^2)$$

Total time: O(n^2)

Space Complexity:

Recalling back to the memoization data structure section of this document, we would be using 2 1-D arrays, one for optimal cost and the other for line-break option that got us that cost. The total space should be O(n)

f. **Write down the algorithm.** You know what order to consider the subproblems, and you know how to solve each subproblem. So do that! If your data structure is an array, this usually means writing a few nested for-loops around your original recurrence, and replacing the recursive calls with array look-ups.

In the form of pseudocode:

```
splitWords(lengths, L, sf) {
    n = lengths.length

    For i = 0 to n - 1 {
        If lengths[i] > L:
            Return null

    Create M[0 to n]
    Create solutions[0 to n]

    M[n] = 0

    For i = n - 1 to 0:
        M[i] = infinity
        width = 0

        For j = i to n - 1:
            Width = width + lengths[j]

            If j > i:
                Width = width + 1

            If width > L:
                Break

            Slack = L - width
            Weight = sf(slack) + M[j+1]

            If weight < M[i]:
                M[i] = weight
                Solutions[i] = j
```

Eric Romero
CSC 445
3/30/2026

```
Create empty list: output
I = 0
While i < n:
        Output.add (solutions[i])
        I = solutions[i] + 1

Return output list
```